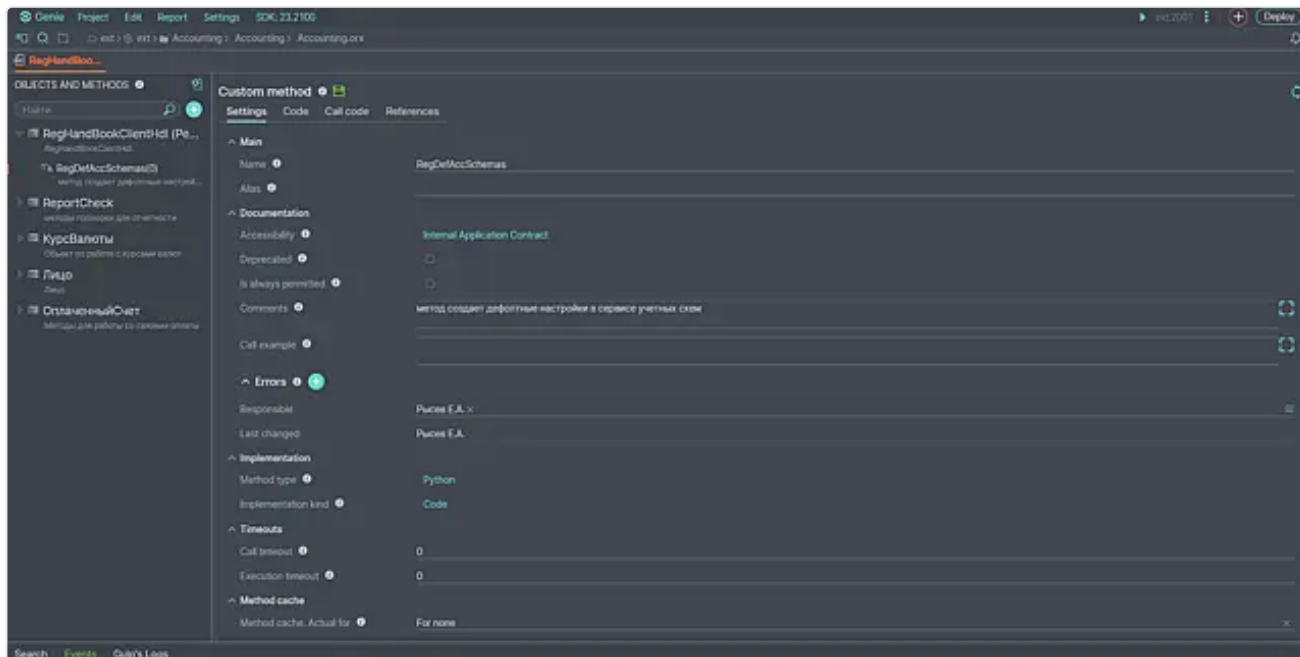


Метод с произвольным количеством параметров (Custom method)

Произвольный метод (или метод с произвольным количеством параметров) имеет произвольную сигнатуру, то есть может принимать любое число параметров и возвращать список записей, запись, отдельный скаляр любого формата либо не возвращать ничего.



Произвольные методы используют в тех случаях, когда стандартная реализация или возможности конфигурации одного из типов платформенных методов не подходят для решения поставленной задачи.

Реализовать методы данного типа можно либо как функцию в библиотеке на Python или C++, либо как SQL-запрос.

Для работы Custom method в зависимости от типа реализации (см. [Method type](#)) необходимо, чтобы в состав сервиса приложения были подключены следующие [сервисные модули](#):

- при реализации метода на C++ (Native function) — сервисный модуль [BL Core](#).
- при реализации метода на Python — сервисный модуль "BL Python".
- при реализации метода на SQL Request — сервисный модуль "Бизнес-логика".

Перечисленные сервисные модули поставляются в составе [SBIS SDK](#).

Методы с типом реализации SQL Request

Методы, реализуемые на SQL могут передаваться как чистое SQL-выражение в двух синтаксисах:

1. С именованными параметрами.

```
1 INSERT INTO "ТестФормата"( "UUID" )
2 VALUES( :UUID )
```

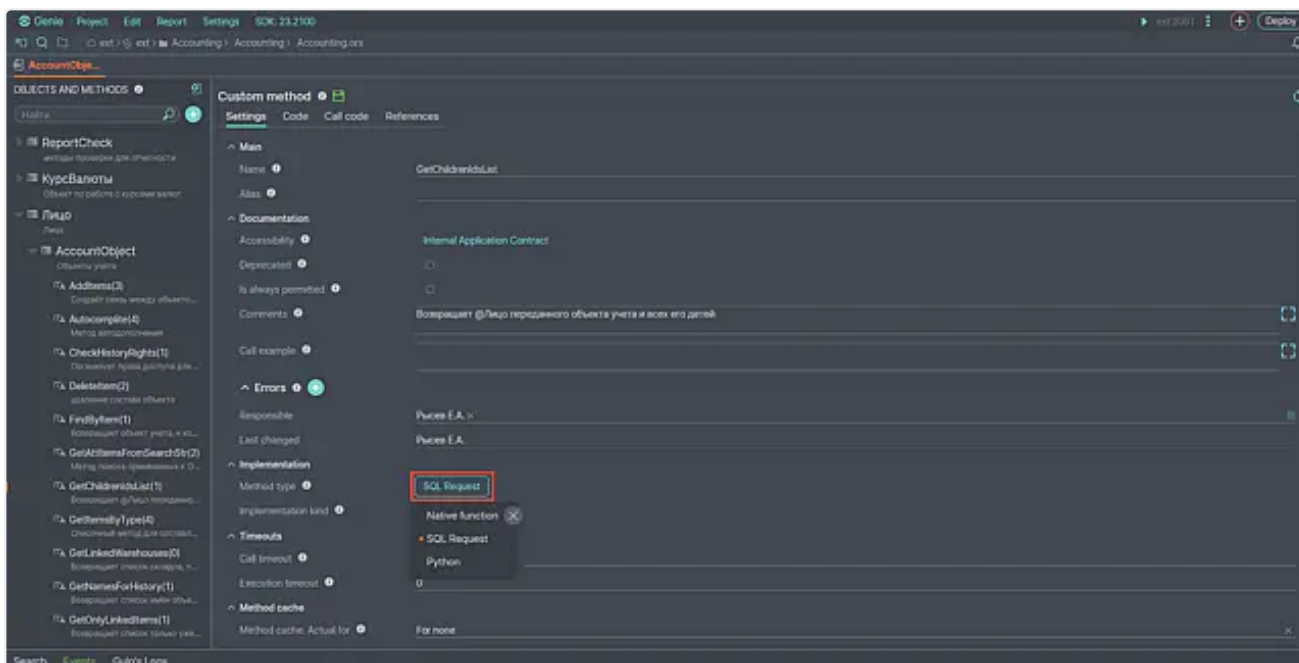
2. С нумерованными параметрами.

```
1 INSERT INTO "ТестФормата"( "UUID", "Целое2", ... )
2 VALUES($1, $2, ...)
```

Подстановка будет осуществляться по порядку объявления параметров в Genie.

Эти способы взаимоисключающие, т.е. в конкретном методе должен применяться только один из синтаксисов.

Параметры типа Record передаются как hstore.



Работа с конфиденциальными данными

В методах, работающих с информацией не допустимой к разглашению (логинами, паролями, персональными данными пользователей и пр.), очень важно не допустить случайную утечку конфиденциальных данных. Такие данные могут приходить в метод в качестве аргументов вызова, использоваться методом в запросах в БД, передаваться им во вложенные вызовы. А типовым сценарием работы метода БЛ является логирование его вызова с указанием аргументов и логирование производимых им запросов в БД и их аргументов.

Если конфиденциальные данные передаются в метод через параметры, необходимо установить опцию [Don't log method input arguments](#). Это установит запрет на логирование аргументов вызова метода и аргументов запросов в БД, исполняемых методом.

Если конфиденциальные данные возвращаются методом в результате, необходимо установить опцию [Don't log method output](#). Это установит запрет на логирование результата вызова метода и аргументов запросов в БД, исполняемых методом.

В фабричных методах, конфиденциальность результата которых становится известна лишь во время исполнения, например, при выборке по ДопПолям, содержащим конфиденциальные данные, необходимо воспользоваться механизмом обозначения конфиденциальности результата RPC-вызова ConfidentialOutput [\[C++/Python\]](#). Это установит запрет на логирование результата текущего RPC-вызова.

- **Python:**

```
sbis.ConfidentialOutput.Enforce()
```

- **C++:**

```
sbis::ConfidentialOutput::Enforce();
```

Если конфиденциальные данные передаются методом во вложенные вызовы неконфиденциальных методов (у которых опция Don't log method input arguments не установлена) необходимо воспользоваться механизмом обозначения конфиденциальной области ConfidentialScope [\[C++/Python\]](#). Это установит запрет на логирование аргументов вызовов локальных методов и аргументов запросов в БД на всю глубину вложенности.

- **Python:**

```
1 with sbis.ConfidentialScope():
2     result = sbis.BLObject('Объект').Invoke('Метод',
3         confidential_data)
```

- **C++:**

```
1 {
2     sbis::ConfidentialScope cs{};
3     Type result = sbis::blcore::Object{ L"Объект" }.Invoke< Type >(
```

```
4 | L "Метод",  
   | confidential_data );  
5 | }
```